

I 주제

1. 개발 배경 및 필요성

(1) GPT가 코딩을 대신 해주는 시대, 왜 우리는 여전히 코딩을 배워야 할까?

- GPT는 강력한 도구이다. 도구를 잘 다루기 위해서는 생각하는 힘이 필요
- GPT는 코딩하는 법을 알려줄 순 있어도 무엇을 만들지 대신 생각하지 못함
- 아이디어를 기획하고 원하는 기능을 명확히 설명해야 GPT가 도와줄 수 있음
- GPT가 만든 코드를 읽고 수정하는 능력은 여전히 사람의 몫임
- 코드를 읽을 줄 모르거나 디버깅을 할 줄 모르면 GPT의 답변을 이해하지 못함
- 코딩은 단순한 기술이 아닌 사고력, 문제해결력을 키우는 과정으로 오류-수정-시도-개선
의 반복에 대한 논리력, 인내력, 분석력이 필요함
- GPT는 보조자일 뿐 창의력은 인간의 영역, GPT는 과거 데이터 기반으로 답변하지만
우리 학생들은 새로운 기능, 앱, 게임 등 처음부터 구상하고 창작해 낼 수 있음
- **미래는 GPT와 함께 코딩을 잘 설명하는 사람이 더 유리함**
- 프로그래밍을 이해하고 설명할 수 있어야 GPT를 제대로 사용할 수 있음

(2) 생각을 언어로 구조화하는 능력인 의미를 이해하고 전달하는 인지능력과 소통 능력이 필요

- 코드의 핵심 기능과 목적을 파악하고 간결한 말로 요약할 수 있는 개념화 능력 필요
- 조건문, 반복문 등 코드의 흐름을 원인과 결과 구조로 설명할 수 있는 논리적 사고력 필요
- 프로그램 실행 흐름을 시간적·절차적으로 나열하며 설명하는 절차적 순서화 능력 필요
- 코드의 문제점을 파악하고 수정 방향을 말로 설명할 수 있는 오류 인식 및 디버깅 감각 필요
- 내가 아는 것과 모르는 것을 구분하고 GPT의 이해 수준에 맞게 설명하는 메타인지와 소통 역량 필요

(3) GPT를 이용한 AI 기반 프로그래밍 학습 플랫폼 필요

- 현재 교육 환경은 여전히 교과서 중심의 수업과 정답 위주의 평가 방식으로 학생들이 실제로 문제를 해결하거나 창의적인 아이디어를 구현하는 경험이 어려움
- GPT는 학생들이 자신의 생각을 직접 표현하고 실시간 피드백을 통해 코딩 과정을 점검할 수 있는 대화형 도구로 활용 가능
- 학생들은 GPT에게 내가 만든 아이디어를 설명하고 그에 대한 실행 가능한 코드 제안을 받으며 사고력과 구현 능력을 동시에 향상 가능
- 학습 플랫폼을 활용한 반복적인 실습 과정을 통해 학생이 자연스럽게 코드 구조, 코드 실행의 흐름, 문제 해결 방법 등을 익힐 수 있음
- GPT의 특징은 수준별 맞춤형 피드백이 가능하다는 것으로 학습자의 이해 수준에 맞는 질문과 설명을 제공해 자기주도 학습 환경을 조성하는데 효과적임
- AI 기반 프로그래밍 학습 플랫폼은 **학생들이 배우는 사람에게 설명하는 사람으로 성장하도록 도와줄 수 있음**
- 이는 미래 AI 활용 시대의 핵심 역량인 설계, 설명, 창의적인 역량이 동시에 키울 수 있는 기회의 장을 제공

2. 주제(목적)

☑ GPT를 활용한 AI 기반 자기주도형 프로그램 학습 플랫폼 개발

- 우리의 플랫폼은 학생들이 정보 교과서에서 배우는 개념을 단순히 수동적으로 암기하는데 그치지 않음
- 학생들이 GhatGPT와의 대화를 통해 스스로 개념을 설명하고 실습 문제를 해결하며

학습 내용을 내면화하도록 유도

☑ 학생들이 GPT를 피드백을 주는 동료처럼 활용할 수 있는 학습 플랫폼 개발

- 자신의 사고 과정을 언어화하고 그 과정에서 코딩의 필요성과 의미를 스스로 체감
- 반복적인 피드백 루프를 통해 학생의 문제 해결력과 디버깅 능력 향상

☑ 기존 교과서 중심 수업에서 벗어나 AI와 상호 작용을 하면서 배우는 능동적이고 창의적인 학습 환경을 제공하는 AI 기반 프로그램 학습 플랫폼을 개발

3. 유사 제품(연구) 및 차별점

(1) 유사 제품 및 연구 사례

- 엔트리(Entry) : 한국형 블록코딩 교육 플랫폼
 - 초·중등 학생 대상의 코딩 학습 콘텐츠 제공
 - 시각적이고 쉬운 구성으로 초보자에게 적합하지만 개별 피드백이나 사고력 기반 질문 유도는 한계
 - <https://playentry.org>
- code.org : 미국 SW 교육 플랫폼
 - 블록 기반 코딩 학습 및 자가 학습형 콘텐츠 제공
 - 개념설명은 있지만 AI와의 대화형 피드백은 불가능함
 - <https://code.org>
- Teachable Machine : Google AI 플랫폼
 - 머신러닝 모델을 시각적으로 만들 수 있는 웹 도구
 - 실습형 콘텐츠 제공에는 적합하나 기초 개념 설명이나 코드 수정 피드백은 제공하지 않음
- 관련 연구
 - AI 튜터: 개인이 참여하는 혁신적 대회 분석
 - ◆ 학생들에게 맞춤형 학습을 제공하여 교사에게는 더 창의적이고 의미있는 수업에 집중할 수 있도록 도와줌
 - ◆ 하지만 학생들이 직접 학습 내용을 설명하고 피드백을 받는 구조는 아님
 - ◆ <https://playtechtrend.com/ai-튜터-개별-맞춤형-학습-혁신-사례-분석/>
 - ChatGPT 활용이 비SW전공생 프로그래밍 학습에 미치는 효과성에 관한 연구
 - ◆ 비SW전공자를 대상으로 한 스크래치 수업에서 ChatGPT의 활용이 학습 성과에 미치는 영향을 조사한 연구
 - ◆ ChatGPT는 스크래치의 고급 기능 이해 및 알고리즘 학습과 같은 난이도 높은 학습 영역에서 뛰어난 효과를 보임
 - ◆ <http://journal.dcs.or.kr/xml/42133/42133.pdf>

(2) 차별점

- 유사 제품 및 관련 연구 : 단순히 코딩 기능 습득이나 GPT를 활용한 교수자의 보조 수단으로 사용, 또한 대부분 일방향 콘텐츠 중심이거나 학습자의 수준에 맞는 피드백이 어려움
- 우리 작품은 ChatGPT와 실시간 대화 기반 상호 작용인 동료 교수법으로 대체
- 피지컬 컴퓨팅 요소(햄스터봇, 아두이노 등)를 활용한 체험 기반 학습을 설계
- 학생들의 메타 인지와 소통 능력 향상 가능
- GPT와의 피드백 루프를 통해 자기주도 학습 및 창의적 문제 해결 능력 강화 가능

II 작품 설계 및 제작

1. 작품 설계 및 목표

(1) 설계 개념

- 우리의 작품은 "학생들이 AI에게 배우는 것이 아니라 AI에게 설명하면서 스스로 배우는 플랫폼"이라는 개념을 중심으로 설계하고자 함

- 단순히 GPT로부터 답을 얻는 수동적인 방식이 아니라 학생이 자신이 이해한 개념을 설명하고 GPT가 이에 피드백을 주는 동료 교수법 기반 학습 구조를 구현하고자 함
- 햄스터봇, 아두이노 등의 피지컬 컴퓨팅 요소를 연동하여 실제로 코드를 입력하고 실행해보는 체험 기반 학습 환경을 제공할 있도록 설계하고자 함

(2) 설계 시 중점을 둔 방향

- 자기주도형 학습 : 학생이 질문자가 아닌 설명자가 되어 GPT와 상호작용함으로써 학습 내용을 내면화
- 실시간 피드백 : GPT API를 활용한 즉각적인 반응과 오류 수정 가이드 제공
- 체험 중심의 실습 환경 : 햄스터봇, 아두이노 등을 통해 실험 활동을 통해 코드와 현실을 연결
- 학습 수준 맞춤화 : GPT가 학생의 수준에 맞는 질문을 유도하고 힌트를 제시하여 흥미를 유도
- 직관적 UI/UX : 초등학생부터 고등학생까지 쉽게 접근 가능하도록 단순하고 시각적인 인터페이스 구성

(3) 주요 기능

- GPT 대화 학습 기능
- 코드 입력 및 실행 창
- 피지컬 컴퓨팅 연동 기능
- 단계별 학습 구조
- 학습 이력 저장 및 피드백 기록

2. 전체 구조 및 시스템 구성도

(1) 플랫폼 전체 흐름도 개요

- 사용자(학생)가 직접 개념을 설명하거나 코드를 작성하고 GPT는 이에 대해 피드백을 주며 학습 콘텐츠는 단계별로 제공되는 학습을 유도
- GPT를 이용해 학생이 작성한 코드는 시뮬레이션 창 또는 피지컬 컴퓨팅 장치와 연동되어 실제 작동 결과를 확인
- 학습 내용 이해, 코드나 설명을 통한 표현, 코드 실행 및 실습, 피드백 과정을 반복적으로 수행하여 개선의 순환 구조

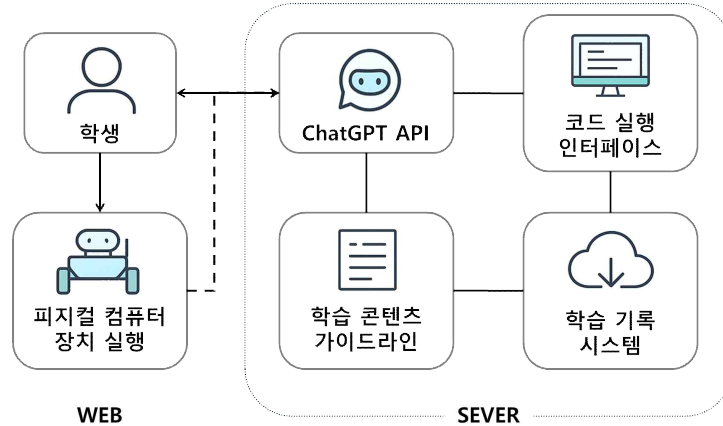
(2) 구성 요소 간 관계 설명

구성요소	역할 및 기능
사용자(학생)	학습의 주체로서 개념을 설명하고 GPT에게 질문하거나 피드백을 받음
GPT(ChatGPT API)	학생의 입력(설명, 코드 등)에 대해 피드백 제공, 질문 유도, 힌트 제공
학습 콘텐츠	단원별 코딩 개념 및 문제 제공 조건문, 반복문, 함수 등 구조화된 구성
코드 실행 시스템	학생이 입력한 코드를 실행 및 시각화하는 브라우저 기반 실행 환경
피지컬 컴퓨팅 요소	햄스터봇, 아두이노 등으로 출력이 연결되어 실제 작동 확인 가능
데이터 저장 및 기록 시스템	GPT 대화 내용, 학습 진도, 코드 실행 이력 등을 자동 저장

(3) 설계 포인트

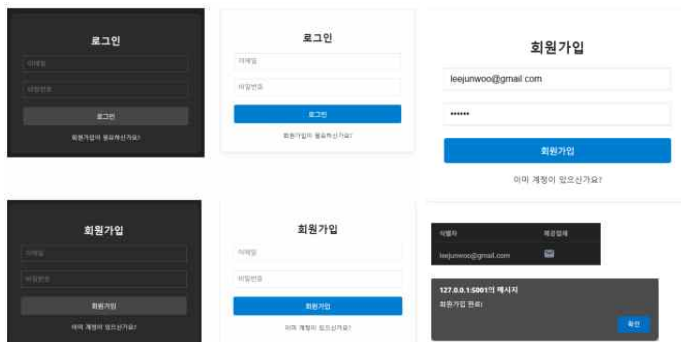
- 양방향 소통 기반 학습 : GPT와 실시간 피드백을 통해 단순 코딩이 아닌 개념 기반 설명력 강화
- 입력-실행-체험의 일체화 : 학생이 코드를 입력하고 실행 결과를 즉시 확인, 피지컬 장치까지 연결
- 개별 맞춤형 학습 경로 : 학습자의 이해 수준, 실수 유형에 따라 GPT가 유도 질문 또는 힌트 제공
- 학습 결과 자동 기록 : 학생의 학습 과정을 저장하고 복습과 교사 피드백 자료로 활용 가능

(4) 전체 시스템 흐름도



3. 주요 기능 및 세부 화면 설계

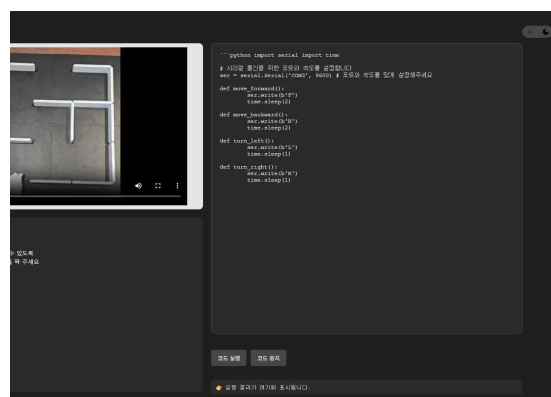
- 로그인/회원가입



먼저 회원가입을 위해 이메일 형식의 아이디와 6자리 이상의 비밀번호를 입력해야한다.

회원가입을 완료하면 파이어베이스DB Authentication에 이메일과 비밀번호가 저장되고 "회원가입 완료!" 라는 메시지가 출력된다.

- GPT 대화
- 코드 입력 및 실행 화면

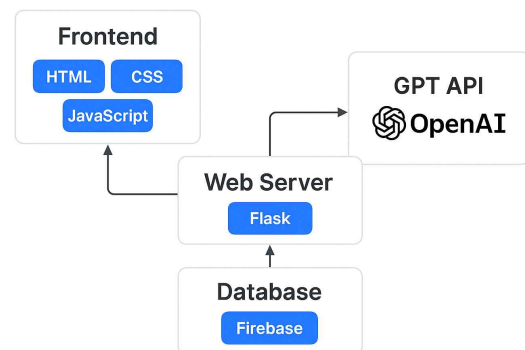


4. 핵심 기술 및 구현 방식

(1) 사용한 기술 스택 구성도

(2) 사용 기술 요약

- 프론트 엔드 [HTML / CSS / JavaScript]
: 사용자 입력 인터페이스 및 결과 출력
- 백엔드 [Python Flask] : 입력 전달, API 연동, 응답 처리, DB 연결
- AI 연동 [OpenAI GPT API]
: gpt-3.5-turbo 또는 gpt-4 모델 호출
- 데이터베이스 [Firebase]
: 사용자 정보, 대화 이력, 학습 데이터 저장 및 조회



5. 현재까지 제작 내용 및 추가 제작 일정

(1) 코딩 보조 교사 관련 주제 및 기획 방향 논의 : 이준우, 김태윤, 배영준

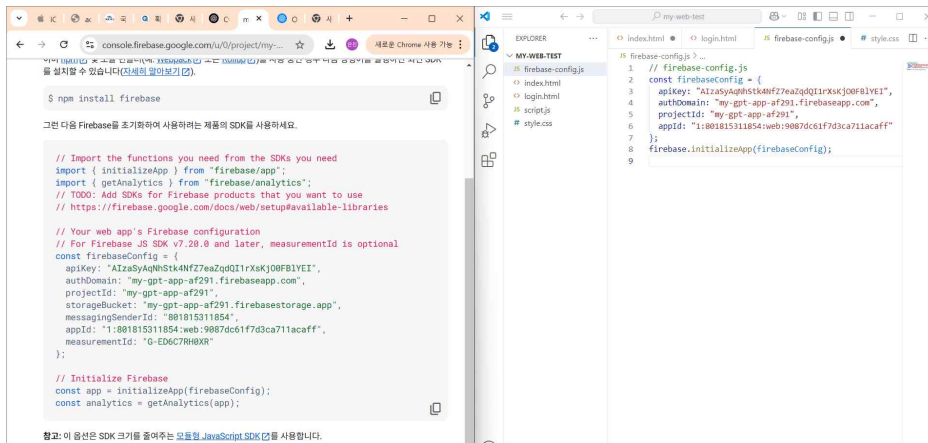
- 코딩 보조교사[로봇 길찾기]
- 길찾기 명령을 주었을 때 로봇이 간 길을 그려서 알려주는 과정에서 학습할 수 있도록
- chatGPT를 이용한 코딩&프로그래밍 보조기능
- chatGPT를 이용한 아두이노 학습 프로그램 : <https://blog.naver.com/bpcode/223623796253>
- 컴퓨터 비전공자를 위한 ChatGPT 기반의 소프트웨어 교양 교육 과정 설계 : <http://journal.dcs.or.kr/xml/39758/39758.pdf>
- 흐름도를 그리면 코드화해주는 프로그램
- 흐름도를 그리는 프로그램 & 자동 코드 작성 및 chatgpt 학습프로그램 : <https://www.youtube.com/watch?v=VNfQNw3w0vI>
- 맵을 그려넣으면 길찾기 코딩 학습이 가능하게 해주는 것

(2) 웹페이지를 이용한 작품 구현을 위한 준비 작업 : 이준우, 김태윤

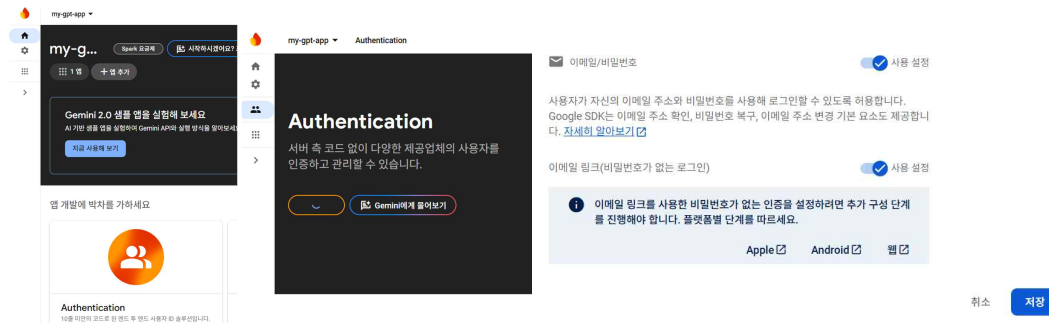
- VS Code 설치
VS Code 다운로드: <https://code.visualstudio.com/>
운영체제에 맞게 설치(Windows/macOS/Linux)
- 추가 플러그인 설치
Live Server : HTML 파일을 브라우저로 실시간 미리보기
Prettier : 코딩 자동 정렬
HTML CSS Support : 문법 강조 기능 제공
JavaScript Language Basics : 자바스크립트 문법 지원
- Test 웹 파일 만들기
/my-web-test
|——index.html
|——style.css
|——script.js
- Live Server 실행
index.html 파일을 열기
우측 아래 또는 우클릭→ **"Open with Live Server"**클릭
기본 브라우저에서 웹페이지가 열림: <http://127.0.0.1:5500/index.html>

(3) 회원가입 및 로그인 구현 : 이준우, 김태윤

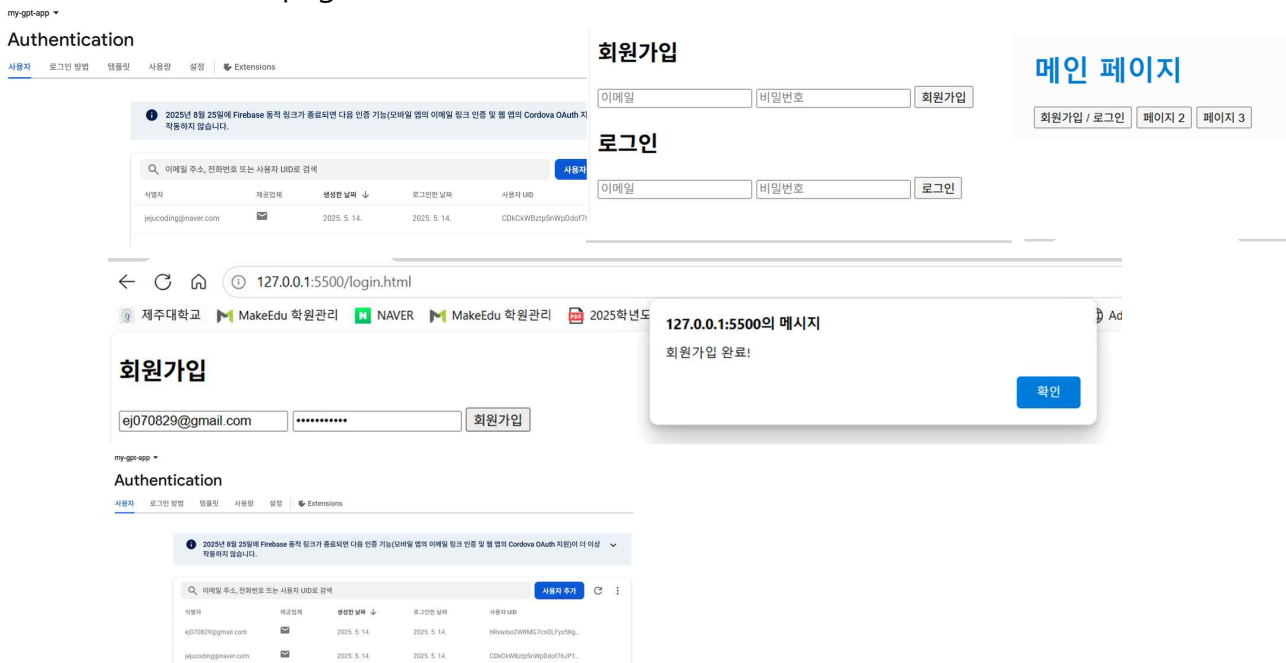
- 프로젝트 구조
/my-web-app/
|——index.html ← 첫 화면
|——login.html ← 로그인/회원가입 페이지
|——page2.html
|——style.css
|——script.js
|——firebase-config.js ← Firebase 연결 설정
- Firebase 웹 프로젝트
 - Firebase 콘솔 열기 : <https://console.firebase.google.com>
 - 웹 앱 등록 : 프로젝트 생성 후 상단 메뉴에서 '</>' 웹 아이콘 클릭
 - 등록 후 나오는 Firebase SDK 설정 코드

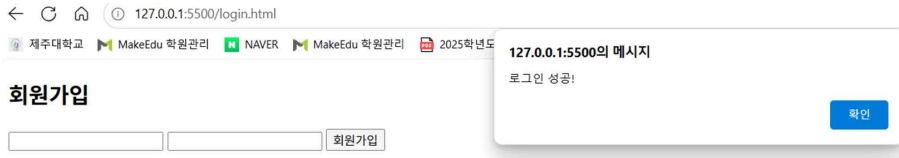


- Firebase 인증(Authentication) 활성화
 Firebase 콘솔 왼쪽 메뉴에서 Authention 클릭
 시작하기 버튼 클릭
 로그인 방법 탭으로 이동
 이메일/비밀번호 : 사용 설정 ON



- 회원가입 및 로그인 기능 구현
 - login.html + Firebase Auth 연결
 - index.html에서 버튼 누르면 다른 페이지 이동 기능 구현
 - login.html에서 Firebase로 회원가입 및 로그인 기능 구현
 - 로그인 후 page2.html로 이동 기능 구현





로그인

ej070829@gmail.com ***** 로그인

페이지 2

[메인으로](#)

(4) 회원가입 및 로그인 화면 디자인 작업 및 자료 정리 : 이준우, 김태윤, 배영준

CSS 원본 파일				
style	2025-05-15 오후 7:27	CSS 원본 파일	4KB	디자인
JSFile				
firebase-config.js	2025-05-14 오후 8:12	JSFile	1KB	자바 스크립트(파이어베이스 연동)
Microsoft Edge HTML Document				
index	2025-05-15 오후 7:16	Microsoft Edge H...	2KB	프론트엔드
login	2025-05-15 오후 7:21	Microsoft Edge H...	3KB	
page2	2025-05-15 오후 7:21	Microsoft Edge H...	2KB	
page3	2025-05-15 오후 7:21	Microsoft Edge H...	2KB	

Microsoft Edge HTML Document				
index	2025-05-15 오후 7:16	Microsoft Edge H...	2KB	
login	2025-05-15 오후 7:21	Microsoft Edge H...	3KB	
page2	2025-05-15 오후 7:21	Microsoft Edge H...	2KB	
page3	2025-05-15 오후 7:21	Microsoft Edge H...	2KB	

index.html : 메인페이지

login.html : 로그인 화면 - 회원가입 및 로그인 가능

page2,3.html : 로그인시 들어가는 화면 - 현재 메인화면으로 돌아가는 기능만 구현



```

const login = () => {
  const email = document.getElementById('login-email').value;
  const password = document.getElementById('login-password').value;
  auth.signInWithEmailAndPassword(email, password)
    .then((user) => {
      alert('로그인 성공!');
      location.href = 'page2.html';
    })
    .catch((error) => alert(error.message));
};

const signup = () => {
  const email = document.getElementById('signup-email').value;
  const password = document.getElementById('signup-password').value;
  auth.createUserWithEmailAndPassword(email, password)
    .then((user) => {
      alert('회원가입 완료!');
      location.href = 'page2.html';
    })
    .catch((error) => alert(error.message));
};
  
```

login.html

텍스트박스와 입력된 이메일과 비밀번호로
Firebase에 새 사용자 계정을 생성한다.
성공 시 "회원가입 완료!" 라는 메시지를 출력하고
실패 시 에러 메시지를 출력한다.

입력된 이메일과 비밀번호로 Firebase 로그인을 요청하고
로그인 성공 시 page2.html로 이동한다.
실패 시 에러 메시지 출력한다.



파이어 베이스에 사용자 추가

login.html 화면에서
회원가입 입력란에
아이디와 패스워드를 입력하고
회원가입 버튼을 누르면
회원가입 완료! 라는 메시지와 함께
파이어베이스에 아이디가 저장된다



```

const firebaseConfig = {
  apiKey: "AIzaSyC1h77idIQULXcTRYoTrLMztQFuQxnl0s",
  authDomain: "my-gpt-app-7a798.firebaseio.com",
  projectId: "my-gpt-app-7a798",
  appId: "1:1039508580470:web:b0e4d4be2d4ed89829757",
};

firebase.initializeApp(firebaseConfig);
  
```

firebase-config.js
파이어베이스와 연동하기위한 준비작업(설정)이다..
사용할 파이어베이스의 api키, 도메인, 프로젝트 아이디
앱 아이디를 받아와
파이어베이스와 연동을 하기위한 설정을 한다.



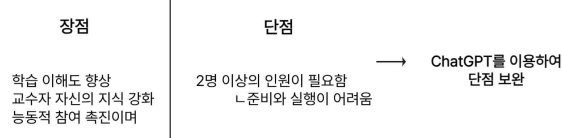
- (5) 아이들이 ChatGPT에게 가르치며 스스로도 아두이노를 배울 수 있는 웹페이지 제작 관련 구상 회의 및 논의 : 이준우, 배영준
- 5단계 아두이노 과정, ChatGPT에게 설명하는 방식, 실습 퀴즈를 포함하여 기능 추가
 - 각 단계의 페이지에 간단한 설명, 코드 예시, 회로 이미지 제공 등 기능
 - 실습 퀴즈 : OX 퀴즈나 빈칸 채우기, 객관식 문제
 - GPT에 설명해보는 입력창을 이용해 설명하면서 답을 찾는 기능
 - 5단계 구조 제안
 - 1단계 : 아두이노란 무엇인가? 마이크로컨트롤러 소개 및 실제 사용 예시
 - 2단계 : LED 켜기 - 회로 구성 및 이론적 내용, 간단한 코드 제시
 - 3단계 : SW 제어 - 회로 구성 및 SW 이론적 설명, 간단한 코드 제시
 - 4단계 : 다양한 센서 사용하기 : 조도 센서, 초음파 센서, 온도 센서 활용
 - 5단계 : ChatGPT에게 설명해보고 내가 배운 내용을 GPT에게 설명하고 피드백 받기
 - ChatGPT 연동 구현 및 실습형 구조의 웹 페이지 구상
 - 각 단계마다 step1.html, step2.html ... 페이지 만들어서 각 페이지에 개념 설명, 실습 코드, 회로 이미지, GPT와의 대화창 기능 추가
- (6) 로그인 로그아웃 기능 구현 : 이준우, 김태윤
- 사용자가 로그인된 상태인지하여 자동으로 로그인 상태 유지
 - 로그인 상태일 때만 특정 페이지 접근 허용
 - 로그아웃 버튼 추가 및 로그아웃 시 자동으로 로그인 페이지로 이동
 - 로그인 상태 감지 코드 : `onAuthStateChanged`
 - 사용자의 로그인 상태를 실시간으로 감지해 자동으로 리디렉션 할 수 있음
- ```
const auth = firebase.auth();
// 로그인 상태 체크
auth.onAuthStateChanged(user => {
 if (!user) { ... } });
// 로그아웃 기능
function logout() { auth.signOut().then(() => { ... }); }
```
- 로그아웃 버튼 추가 : page2.html
 

```
<button onclick="logout()">로그아웃</button>
```
  - 로그인 페이지에서 로그인 상태인 경우 자동 리디렉션
 

```
auth.onAuthStateChanged(user => {
 if (user) { // 이미 로그인된 사용자면 바로 이동 } });
```
- (7) 웹페이지 구상 계획 : 이준우, 김태윤, 배영준

## 동료교수법

학습자들이 서로 가르치고 돕는 협력 학습 방법



## 동료교수법

기존의 컴퓨터교육은 학습자 간 능력과 학습 속도 차이가 큼  
모든 학생에게 맞는 수업을 하려면 많은 시간이 필요함  
이에 동료교수법을 활용하면 효과적임

↓  
ChatGPT에게 가르치며 스스로 아두이노를 배우는 웹페이지 제작

효과적 운영을 위해 명확한 목적 설정, 계획된 교수과정, 의사소통, 동료교수자 훈련 및 지원이 필요함



## 목적

chatGPT를 가르치며 스스로 하드웨어를 배우는 웹페이지 제작

기본적인 구성 요소들

↳ 대상, 학습할 요소, 학습 순서 등



## 기본적인 구성 요소

대상 : 초등~중학생 (AI, 코딩, 아두이노 입문자)

학습할 요소 : step 1 : 햄스터 봇 제어, step 2 : 아두이노

학습 구조 : 단계별 학습, chatGPT에게 설명, 여러 퀴즈

핵심 - 학습자 주도 학습, \*동료 교수 법\*

### (8) 웹 페이지에 GPT 동작 구현 : 이준우, 김태운

- GPT API 연결 : <https://platform.openai.com/account/api-keys>에서 API 키를 발급
- chatgpt - web/ 디렉토리 구조
  - ├── gpt\_test.html
  - ├── server.py
  - └── requirements.txt
- gpt\_test.html : GPT 입력 및 응답 표시

```
10 <button onclick="sendToGPT()">GPT에게 보내기</button>
11 <h3>GPT 응답:</h3>
12 <div id="gptResponse"></div>
13
14 <script>
15 const chatHistory = [
16 { role: "system", content: "너는 초등학교생에게 아두이노를 설명해주는 친절한 선생님이야." }
17];
18
19 async function sendToGPT() {
20 const input = document.getElementById("userInput").value;
21 chatHistory.push({ role: "user", content: input });
22
23 async function sendToGPT() {
24 const input = document.getElementById("userInput").value;
25 chatHistory.push({ role: "user", content: input });
26
27 const response = await fetch("/chat", {
28 method: "POST",
29 headers: { "Content-Type": "application/json" },
30 body: JSON.stringify({ messages: chatHistory })
31 });
32
33 const data = await response.json();
34 const reply = data.reply.content;
35
36 chatHistory.push({ role: "assistant", content: reply });
37
38 document.getElementById("gptResponse").innerText = reply;
39 }
40 }
41 </script>
42 </body>
43 </html>
```

- server.py : Flask 백엔드와 OpenAI 연동

```
server.py / chat
1 from flask import Flask, request, jsonify, send_from_directory
2 from openai import OpenAI
3 import os
4
5 app = Flask(__name__, static_url_path='')
6
7 client = OpenAI(api_key="sk-proj-_h9ggniaE_ZKWas6VNDFHICYuTuHwCGRaCfa_vU-i8-46a4BXTxPn")
8
9 @app.route('/')
10 def index():
11 return send_from_directory('.', 'gpt_test.html')
```



## (10) ChatGPT 웹 UI 및 코드 분석 : 이준우, 김태윤, 배영준

### ChatGPT Arduino 학습

GPT에게 보내기

GPT 응답:

### ChatGPT Arduino 학습

GPT 응답:

LED와 아두이노에 연결한 LED는 LED의 핀 다리를 1번 핀과, 밝은 다리를 GND에 연결하여 줍니다. digitalWrite 함수를 사용하여 LED를 켜고 끄고 delay 함수를 이용하여 적절한 시간 간격을 제어할 수 있습니다. 간단한 문법을 사용하여 LED를 연결하고 켜고 끄는 코드를 작성하여 줍니다.

```
from flask import Flask, request, jsonify, send_file
from openai import OpenAI

app = Flask(__name__, static_url_path='')

client = OpenAI(api_key="sk-proj-_h9gniaE_ZKMas6VNDHICyUtuKwGRCaFv_U-18-46a4BXTxPho8eBplSIVjaVzuovzthjT3B1bkfJv5_U4V1j_wP76H")

@app.route('/')
def index():
 return send_file('gpt_test.html')

@app.route('/chat', methods=['POST'])
def chat():
 data = request.get_json()
 messages = data['messages']

 try:
 response = client.chat.completions.create(
 model="gpt-3.5-turbo",
 messages=messages
)
 return jsonify({'reply': {'content': response.choices[0].message.content}})
 except Exception as e:
 return jsonify({'reply': {'content': f"▲ 에러 발생: {str(e)}"}})

if __name__ == '__main__':
 app.run(debug=True, port=5001)
```

브라우저 접속시 gpt\_test.html 열고  
사용자가 질문 작성 후  
"GPT에게 보내기" 클릭  
자바스크립트가 /chat으로 메시지를 전송한다.  
Flask 서버는 OpenAI GPT에  
메시지를 전달하고 응답을 받아  
사용자에게 돌려줌  
HTML 페이지가 응답을 받아 화면에 표시한다.

```
if __name__ == '__main__':
 app.run(debug=True, port=5001)
```

초기엔 위의 코드가 없어서 주소가  
http://127.0.0.1:5000/  
이여서 웹페이지가 열리지 않았다

## (11) ChatGPT를 우리 웹페이지에 연동 : 이준우, 김태윤, 배영준

```
function sendMessage() {
 const userMessage = chatInput.value.trim();
 if (userMessage) return;

 addMessage(userMessage, "user");
 chatInput.value = "";

 fetch("http://127.0.0.1:5001/chat", {
 method: "POST",
 headers: {
 "Content-Type": "application/json"
 },
 body: JSON.stringify({
 messages: [{ role: "user", content: userMessage }]
 })
 })
 .then(res => res.json())
 .then(data => {
 const reply = data.reply.content;
 addMessage(reply, "bot");
 })
 .catch(err => {
 console.error(err);
 addMessage("▲ 오류가 발생했습니다.", "bot");
 });
}

chatInput.addEventListener("keydown", e => {
 if (e.key === "Enter") sendMessage();
});
</script>
</body>
</html>
```

page2.html의 사이드 바에  
챗봇 UI를 넣는 코드이다.

my-web-test

- venv
- firebase-config.js
- index.html
- login.html
- moon.png
- page2.html
- page3.html
- requirements.txt
- server.py
- style.css
- sun.png
- chatgpt-web
- venv
- venv
- gpt\_test.html
- requirements.txt
- server.py

기존의 웹페이지에  
ChatGPT 테스트를 위해 제작했던  
server.py와 requirements.txt를 가져왔다.

### 테스트용 server.py

```
from flask import Flask, request, jsonify, send_file
from openai import OpenAI

app = Flask(__name__, static_url_path='')

client = OpenAI(api_key="sk-proj-_h9gniaE_ZKMas6VNDHICyUtuKwGRCaFv_U-18-46a4BXTxPho8eBplSIVjaVzuovzthjT3B1bkfJv5_U4V1j_wP76H")

@app.route('/')
def index():
 return send_file('gpt_test.html')

@app.route('/chat', methods=['POST'])
def chat():
 data = request.get_json()
 messages = data['messages']

 try:
 response = client.chat.completions.create(
 model="gpt-3.5-turbo",
 messages=messages
)
 return jsonify({'reply': {'content': response.choices[0].message.content}})
 except Exception as e:
 return jsonify({'reply': {'content': f"▲ 에러 발생: {str(e)}"}})

if __name__ == '__main__':
 app.run(debug=True, port=5001)
```

### IPTeacher의 server.py

```
from flask import Flask, request, jsonify, send_file
from openai import OpenAI
from flask_cors import CORS

app = Flask(__name__, static_folder='.', static_url_path='')
CORS(app, resources={r"//*": {"origins": "http://127.0.0.1:5001"}})
app = Flask(__name__, static_folder='.', static_url_path='')

client = OpenAI(api_key="sk-proj-_h9gniaE_ZKMas6VNDHICyUtuKwGRCaFv_U-18-46a4BXTxPho8eBplSIVjaVzuovzthjT3B1bkfJv5_U4V1j_wP76H")

@app.route('/')
def index():
 return send_file('index.html') # 백인으로 연결

@app.route('/chat', methods=['POST'])
def chat():
 data = request.get_json()
 messages = data['messages']

 try:
 response = client.chat.completions.create(
 model="gpt-3.5-turbo",
 messages=messages
)
 return jsonify({'reply': {'content': response.choices[0].message.content}})
 except Exception as e:
 return jsonify({'reply': {'content': f"▲ 에러 발생: {str(e)}"}})

if __name__ == '__main__':
 app.run(debug=True, port=5001)
```

```
from flask_cors import CORS

app = Flask(__name__, static_folder='.', static_url_path='')
CORS(app, resources={r"/*": {"origins": "http://127.0.0.1:5001"}})

client = OpenAI(api_key="sk-proj-_h9ggniaE_ZKWas6VNDHFHICYuTuHwCGRaCf")

@app.route('/')

```

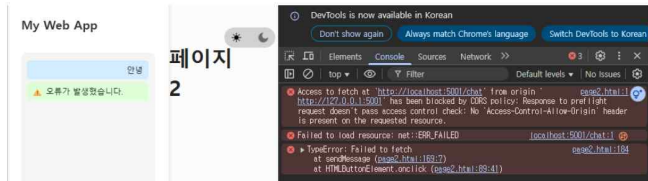
브라우저 보안 정책 때문에 다른 도메인에서 API 호출을 못 하게 막는게되어 있는데 이 설정을 하면 localhost:5001에서 오는 요청을 허용할 수 있게 해준다.

이 두가지 코드가 추가되었다.

```
from flask_cors import CORS
```

```
CORS(app, resources={r"/*": {"origins": "http://127.0.0.1:5001"}})
```

이 코드들을 추가 하지 않으면 에러가 나는데 이 에러는 브라우저가 http://127.0.0.1:5001 (page2.html이 실행 중인 도메인)과 http://localhost:5001 (API 요청 대상)의 출처가 다르다고 판단해 요청을 차단 을 해서 난 에러이다.



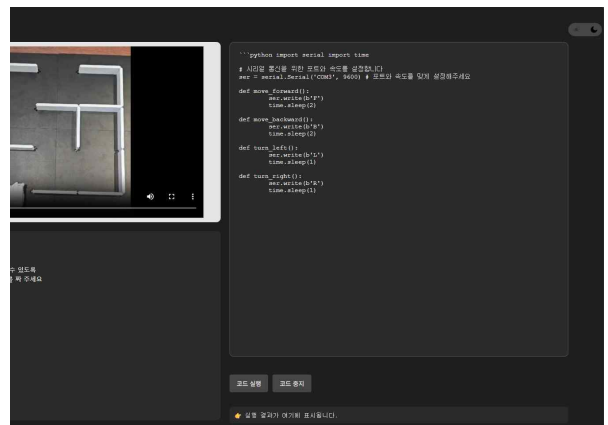
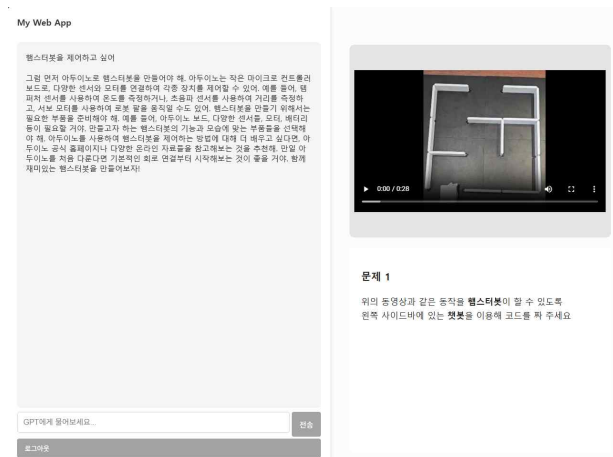
다크모드와 라이트모드 둘다 UI가 잘 적용되었다.

대화내용이 저장되어서 페이지를 넘겼다 다시 돌아와도 전에있던 대화내용을 볼수있는 기능이 구현되지 않았다.

이 문제점은 파이어베이스DB를 통해 개선할 예정이다.

## (12) 웹 화면 구성 완료 및 테스트 : 이준우, 김태윤

- chatGPT 검색 및 코드 실행 화면 구성 및 동작 확인



## (13) 햄스터봇을 이용한 교육 내용 개발 : 이준우, 김태윤

- 햄스터봇 제어 및 교육 내용 개발
- 햄스터봇 설치 및 시작 방법에 대한 교육 내용
- 햄스터봇 제어 및 명령어에 대한 교육 배용
- chatGPT를 이용해 학습 방법 및 UI 구성



햄스터봇을 연결하기 전 화면이다.

햄스터봇을 연결하기전엔  
하드웨어를 연결할수있는 방법들을 알려준다.



"연결 프로그램 열기"를 처음 누르면  
연결 프로그램을 다운로드 해야 한다.



운영 체제에 맞는 버전을 다운로드  
exe.파일을 실행해 엔트리 하드웨어라는 프로그램을 설치한다.

엔트리 하드웨어를 실행하고  
햄스터를 선택을 하고 햄스터봇을 연결한다.



다음은 햄스터봇을 연결하는 방법이다.



동글을 노트북에 연결한다.  
그다음 햄스터봇을 실행하고  
파란불이 들어온다면  
연결이 된것이다.



햄스터봇을 연결하면  
제어를 할 수 있는 블록들과  
햄스터봇의 현재 상태창이 생성된다.

## 모터

## 라인트레이서



햄스터봇의 바퀴(모터)를  
제어하는 코드들이다.  
왼쪽 바퀴, 오른쪽 바퀴 각각  
속도와 구동 시간을 조절 할 수 있다.

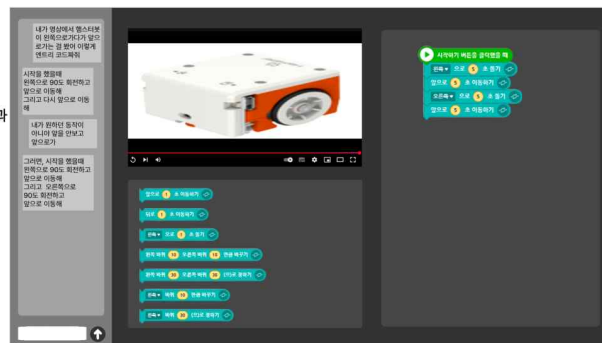


햄스터봇의 라인트레이서를  
사용하는 코드이다.  
선의 색(검은색, 하얀색)과  
오른쪽,왼쪽 센서를 각각 제어가능하다.  
속도를 제어 할 수 있다.

## 기타



블록 앞에 사물이 있는지 감지하거나  
방향별 기울었는지 판단, 배터리 부족 여부 등등 조건 블록과  
각 센서별(조도, 가속도, 근접 등등) 값이 들어있는  
변수형식의 블록도 있다.



## III 작품 테스트 계획

### 1. 테스트 목적

- GPT 연동, 코드 실행, 피지컬 컴퓨팅 연동 등 주요 기능이 기획 의도대로 동작하는지 확인
- 플랫폼의 UI/UX가 사용자(학생)에게 직관적이고 원활한 학습 흐름을 제공하는지 검토



- 사용자가 플랫폼을 통해 사고력, 설명력, 디버깅 능력 등 핵심 역량을 실제로 향상시키는지 확인
  - : 사용자가 '설명자' 역할을 수행하며 자기주도적으로 학습하는 구조가 효과적인지 평가
2. 테스트 방법
- 전체 시스템 기능 점검, 교육 전문가의 교육적 유용성 및 학습 효과 분석, 학생 사용자의 실제 사용성 및 학습 변화 피드백 제공
  - 주요 기능별 테스트 : GPT 대화 학습 시스템, 코드 입력 및 실행 모듈, 피지컬 컴퓨팅 연동(햄스터봇, 아두이노), 학습 이력 저장 및 피드백 제공 기능
  - 사용자 행동 흐름 테스트 : 설명-코드 작성-실행-피드백-개선의 순환 구조, 직관적 인터페이스를 통한 접근 용이성
  - 학습 효과 평가 : 사전·사후 개념 이해도 및 디버깅 설명력 변화 측정, GPT 대화 내용을 통한 논리적 사고력, 설명력 분석
3. 테스트 도구 및 절차
- 관찰 및 녹화 : 학생이 플랫폼을 사용하는 과정 녹화하여 학습 흐름 분석
  - 사용자 설문지 및 인터뷰 : 사용성 만족도, 개선 의견 수집
  - 사전-사후 테스트 : 간단한 문제 풀이 및 설명 테스트를 통해 학습 효과 비교
  - 로그 분석 : 학습 이력, GPT 대화 내용, 코드 수정 패턴 등을 분석해 개선 방향 도출
4. 사용 도구
- Loom/OBS : 사용자 사용 과정 녹화 ,
  - Google Forms : 사용자 피드백 수집
  - Pandas와 Matplotlib : 로그 데이터 정량 분석
  - Flask와 SQLite : 대화 및 코드 이력 저장 및 분석

## IV 제작 현황 및 향후 발전 방안

### 1. 제작 현황

항목	제작 내용 및 진척도
GPT 연동 대화 학습 시스템	gpt-3.5-turbo API 연동 완료, 사용자 입력을 이용해 GPT 응답 구조 구현됨
코드 입력 및 실행 시스템	HTML, JS 기반 코드 에디터 구현, Flask 백엔드와 연동하여 코드 실행 가능
학습 흐름 구성	개념 설명, GPT 피드백, 코드 실행, 수정의 흐름 구현 완료
피지컬 컴퓨팅 연동	아두이노 예제 일부 구현, 햄스터봇 연결 구조 설계됨
회원가입 및 로그인 기능	Firebase 인증 시스템 구현 완료
UI 설계	GPT 대화창, 코드창, 결과창 등 주요 화면 구성 완료
데이터 저장 기능	학습 이력 저장 구조 설계 완료, 기록 기능 구현 중

### 2. 추후 개선사항 : 결론에 따라 추가적인 개선점을 기술

항목	현재 진행 상태	향후 계획 및 개선 방향
피지컬 컴퓨팅 연동	아두이노 코드 일부 구현 햄스터봇 연결 설계됨	다양한 센서 및 출력 예제 확대, 실기 기반 연동 테스트 강화
GPT 피드백 구조	기본 대화 구조 구현	오류 지점 인식 및 수준별 피드백 강화, 설명 유도형 prompt 개선

학습 흐름 콘텐츠	개념 설명 실행 흐름 구현	조건문, 반복문 등 단원별 단계 콘텐츠 보완, 실습 흐름 세분화
사용자 인터페이스 (UI/UX)	기본 화면 구현	직관적 구성 개선, 학년별 맞춤형 인터페이스(폰트, 버튼, 레이아웃) 개선
데이터 저장 및 분석	코드 입력 기록 저장 구조 설계 중	대화, 실행, 수정 히스토리 저장 사용자 맞춤 피드백 제공 기반 구축
회원 시스템	로그인 회원가입 기능 구현	사용자별 대시보드, 학습 진도 기반 콘텐츠 제안 기능 추가
테스트 계획 (기능 검증)	내부 테스트 준비 중	GPT 응답 정확성, 코드 실행 정확성, 시뮬레이션 정상 작동 여부 확인
테스트 계획 (사용자 효과 검증)	학습자 피드백 수집 계획 수립	사전·사후 평가, 사용 로그 분석, 설문 및 인터뷰를 통한 학습 효과 측정
개선 피드백 적용	초기 사용자 관찰 계획 중	피드백 기반 학습 흐름 조정 및 설명 방식 개선 예정

## 출처 표기

- 엔트리(Entry) : 한국형 블록코딩 교육 플랫폼  
- <https://playentry.org>
- code.org : 미국 SW 교육 플랫폼  
- <https://code.org>
- Teachable Machine : Google AI 플랫폼  
- <https://teachablemachine.withgoogle.com/>
- AI 튜터: 개인이 참여하는 혁신적 대회 분석  
- <https://playtechtrend.com/ai-튜터-개별-맞춤형-학습-혁신-사례-분석/>
- ChatGPT 활용이 비SW전공생 프로그래밍 학습에 미치는 효과성에 관한 연구  
- <http://journal.dcs.or.kr/xml/42133/42133.pdf>
- ChatGPT  
- <https://chatgpt.com/>
- AI 연동 [OpenAI GPT API]  
- <https://openai.com/>
- chatGPT를 이용한 아두이노 학습 프로그램  
- [https://blog.naver.com/bpcode/223623\\_796253](https://blog.naver.com/bpcode/223623_796253)
- 컴퓨터 비전공자를 위한 ChatGPT 기반의 소프트웨어 교양 교육 과정 설계  
- <http://journal.dcs.or.kr/xml/39758/39758.pdf>
- 흐름도를 그리는 프로그램 & 자동 코드 작성 및 chatgpt 학습프로그램  
- <https://www.youtube.com/watch?v=VNfQNw3w0vI>
- Firebase DataBase  
- <https://firebase.google.com>
- 아두이노 소프트웨어&하드웨어  
- <https://www.arduino.cc/en/software/>